instituto superior de engenharia de lisboa

DiárioLX — Plataforma Digital de Jornalismo Académico

Jessé Alencar
Julianne Lobato

Orientadores: *Artur Ferreira*
Filipe Freitas
Fátima Cardoso (ESCS)

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

Instituto Superior de Engenharia de Lisboa

DiárioLX — Plataforma Digital de Jornalismo Académico

51745 Jessé Alencar

51191 Julianne Lobato

Orientadores: Artur Ferreira

Filipe Freitas

Fátima (ESCS)

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

Resumo

Resumo do projeto.

Usage of Generative AI Tools

Throughout this project, generative artificial intelligence tools were used to support different stages of the development process. These tools were primarily employed for brainstorming technical solutions, exploring architectural approaches, assisting with documentation writing and structuring, and supporting the initial prototyping of certain interface components.

AI tools were also used to generate preliminary code examples, assist in the creation of Bootstrap-based layouts, and clarify technical concepts related to the technologies adopted in the project.

All final architectural, implementation, and validation decisions were made by the project authors.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context

In a media ecosystem increasingly marked by misinformation and the erosion of journalistic standards, Diário LX emerges as a digital journalism initiative committed to editorial ethics, slow journalism, and proximity to local communities. The project is developed within the Journalism Trends Laboratory (*Laboratório de Tendência no Jornalismo*), a structure affiliated with LIACOM (Applied Research Laboratory in Communication and Media) at the Escola Superior de Comunicação Social (ESCS) of the Instituto Politécnico de Lisboa (IPL).

The platform targets a young audience aged 18 to 35 years, with higher education backgrounds, and places a particular emphasis on the Greater Lisbon region. Editorially, it promotes visual journalism, long-form reporting, and investigative content, while deliberately avoiding clickbait-driven content strategies. The project has no commercial purpose and carries no advertising, operating instead with civic, educational, and scientific goals, co-funded by FCT (Fundação para a Ciência e Tecnologia).

1.2 Motivation

A proof of concept was initially implemented using WordPress to validate the editorial structure and workflow. However, this approach revealed significant limitations in terms of architectural control, customisation, and long-term scalability. The need to build a purpose-built platform that fully supports the editorial lifecycle — from content creation and review to publication and archival — motivated the development of this system from the ground up.

1.3 Solution Overview

The resulting system, Diário LX, is a web-based content management and publication platform organised around two distinct areas:

- **Backoffice**: A restricted editorial management interface, accessible only to authenticated users (Administrators, Editors, and Contributors), supporting content creation,

revision, approval, and publishing workflows.

- **Public Website**: A reader-facing interface providing access to published articles, multimedia content (podcasts, videos, photo essays), organised by category, tag, and editorial section.

The system enforces a structured editorial workflow in which content progresses through defined states — *draft*, *pending approval*, *published*, and *rejected* — ensuring editorial oversight before any content reaches the public.

1.4 Report Organization

The remainder of this report is organised as follows:

- Chapter 2 presents the context, related work, and existing solutions analysed during the project.
- Chapter 3 details the functional and non-functional requirements of the system.
- Chapter 4 describes the system architecture and technology choices.
- Chapter 5 presents the data model.
- Chapter 6 covers the backend implementation.
- Chapter 7 covers the frontend implementation.
- Chapter 8 describes the testing approach and results.
- Chapter 9 concludes the report and outlines future work.

Chapter 2

Background

2.1 Digital Journalism Platforms

Digital journalism has undergone significant transformations in the past decade, driven by the proliferation of social media, mobile devices, and algorithmic content distribution. Traditional newsrooms have increasingly migrated to web-native platforms, yet many existing solutions prioritise speed and volume over editorial rigour and journalistic depth [1].

Slow journalism, as a counter-movement, emphasises long-form reporting, investigative depth, and contextual analysis. Platforms designed to support slow journalism require features that differ substantially from high-volume news aggregators: robust editorial workflows, multimedia support, and fine-grained access control for editorial teams.

2.2 Existing Solutions

Several content management systems (CMS) were analysed as potential off-the-shelf solutions before the decision was made to develop a bespoke platform.

2.2.1 WordPress

WordPress is the most widely adopted CMS in the world, powering over 40% of all websites [2]. A WordPress-based proof of concept was the starting point for this project. Although WordPress provides a rich plugin ecosystem and a familiar editorial interface, it presents notable limitations in the context of Diário LX:

- Limited control over the editorial approval workflow without heavy plugin dependencies.
- Performance and security concerns at scale.
- Architectural rigidity that complicates custom API design and frontend decoupling.

2.2.2 Ghost

Ghost is a modern, open-source publishing platform focused on independent journalism and newsletters. It offers a clean editorial interface and native support for membership models.

However, its workflow model does not natively support multi-role editorial approval chains, and its theming system imposes constraints on frontend customisation.

2.2.3 Summary

Table 2.1 summarises the key features of the evaluated solutions in comparison with the requirements of Diário LX.

Table 2.1: Comparison of existing CMS solutions.

Feature	**WordPress**	**Ghost**	**Diário LX**
Editorial workflow	Plugin-based	Limited	Built-in
Role-based access	Plugin-based	Basic	Multi-level
Multimedia support	Plugin-based	Basic	Native
API-first design	Partial	Yes	Yes
Custom frontend	Theme-based	Theme-based	React (decoupled)
Open source	Yes	Yes	Yes

None of the analysed solutions fully satisfies the editorial workflow, access control, and multimedia management requirements of Diário LX. This motivated the development of a purpose-built platform.

Chapter 3

System Requirements

This chapter presents the functional and non-functional requirements of the Diário LX platform, derived from the project specification and validated with the editorial team.

3.1 Functional Requirements

3.1.1 Content Management

The system must provide a full content management lifecycle supporting the following content types: **articles**, **podcasts**, **videos**, and **photo essays**. Each content item must include the following attributes:

- Title
- Body text (supporting inline subtitles, highlights, and captioned images)
- Lead paragraph (*entrada*)
- Author(s) and photo author
- Creation and publication date
- Featured image
- Category and subcategory
- Tags

Content must progress through a defined workflow:

1. **Draft** — created and edited by a Contributor.
2. **Pending Approval** — submitted for editorial review.
3. **Published** — approved and publicly visible.
4. **Rejected** — returned to the author with revision comments.

It must be possible to archive and permanently delete content. Media file uploads are limited to 2 GB. It must be possible to mark content as featured for display on the homepage.

3.1.2 Categories, Subcategories, and Tags

Content must be organised using categories and subcategories. The main editorial sections are:

- Lisboa, Cidade Aberta
- A Fundo
- Secções (Mundo, Política, Sociedade, Economia, Cultura e Media, Bem-Viver, Desporto)
- Especiais
- Fotografia
- Podcasts
- Vídeos

Each category must support a name, an associated colour, and an automatically generated listing page. Tags must also have dedicated listing pages. The system must allow the creation, editing, and deletion of categories and tags.

3.1.3 User Management and Permissions

The system defines three user roles:

Administrator Full system control: manages users, system settings, approves or rejects content and new accounts, and has access to all backoffice areas.

Editor Can approve or reject submitted content, manage contributor accounts, and deactivate users.

Contributor Can create and submit content for approval. Cannot publish directly.

Each user profile includes: name, email, profile photograph, active/inactive status, professional function (journalist or photographer), and role.

3.1.4 Backoffice

The backoffice must provide:

- Account creation for Editors and Contributors.
- Login and logout for all profiles.
- Listing and search of all articles and users.
- Profile editing (own profile for all; any profile for Administrators).
- Comment moderation (approve, reject, hide) by Administrators and Editors.

3.1.5 Public Website

Homepage

The homepage must display:

- 4 featured articles (defined in the backoffice).
- The 4 most recent articles.
- Navigation to main editorial sections.
- Links to institutional pages: Team, Editorial Statute, and Code of Ethics.

Other Pages

- **Category page**: one featured article (defined in the backoffice) and remaining articles in reverse chronological order.
- **Tag page**: list of associated articles.
- **Article page**: full article view, reader comments, and 4 related articles.
- **Team page**.
- **Podcast page**: dedicated layout distinct from other category pages.

3.1.6 Search and Navigation

- Full-text keyword search across articles.
- Human-readable, slugified URLs (e.g. `/politica/orcamento-estado-2026`).

3.1.7 Compatibility

The system must be compatible with modern browsers and fully functional on mobile devices (responsive design for mobile, tablet, and desktop viewports).

3.2 Non-Functional Requirements

- **Security**: Authentication via token-based mechanism; role-based access control enforced at the API level.
- **Performance**: Pages must load within acceptable response times under expected editorial team and reader load.
- **Maintainability**: Codebase organised to allow independent evolution of frontend and backend.
- **Media storage**: Support for images, audio (podcasts), and video files up to 2 GB per upload.
- **Scalability**: Architecture must support growth in both content volume and concurrent users.

3.3 Optional Requirements

The following features are considered optional and may be addressed in future iterations:

- Scheduled publication of content.
- Advanced search filters (by date, category, author, tag) in both backoffice and public site.
- User search filters in the backoffice (by role, function, status).
- Reader account registration and commenting.
- Audiodescription support for visually impaired users.
- Multilingual support (Portuguese and English).

Chapter 4

Architecture

4.1 Overview

The Diário LX platform follows a client-server architecture. The system is composed of a browser-executed client application and a server-side infrastructure responsible for business logic and data management. Figure 4.1 illustrates the high-level architecture of the system.

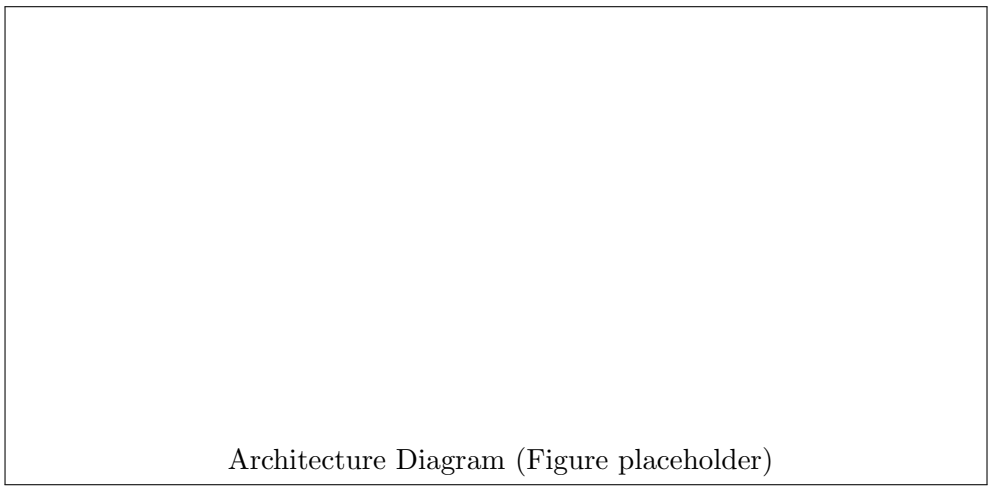

Figure 4.1: High-level system architecture.

The system comprises the following main components:

- **React Client**: A single-page application (SPA) running in the browser, responsible for rendering the public website and the backoffice interface.
- **Reverse Proxy**: Serves static assets and routes HTTP requests to the appropriate backend service.
- **Spring Boot API**: The backend application, developed in Kotlin, responsible for all business logic, authentication, and data access.

- **PostgreSQL Database**: Relational database storing structured data (articles, users, categories, tags).

- **Media Storage**: Dedicated storage system for multimedia files (images, audio, video). The specific provider is to be decided.

4.2 Technology Stack

Table 4.1 summarises the technologies adopted in this project.

Table 4.1: Technology stack.

Layer	**Technology**	**Rationale**
Frontend	React (TypeScript)	Component-based UI architecture and strong ecosystem support
Backend	Spring Boot (Kotlin)	Robust HTTP API development, JVM ecosystem integration, and type safety
Database	PostgreSQL	Relational model well-suited for editorial and multimedia data
Reverse Proxy	Nginx	Standard and performant reverse proxy and static asset serving
Media Storage	SeaweedFS	S3-compatible distributed storage with low operational overhead
Authentication	Bearer Tokens	Server-side token invalidation and simplified session management

4.3 API Design

Communication between the frontend and backend is handled through a RESTful HTTP API. All endpoints are prefixed with `/api/v1/`. The API follows standard HTTP conventions for resource naming and status codes.

4.4 Authentication and Authorization

Authentication is implemented using a token-based mechanism (JWT). Upon successful login, the server issues a signed token that the client includes in subsequent requests via the `Authorization` header. Role-based access control (RBAC) is enforced at the API level: each endpoint validates the token and the caller's role before executing the requested operation.

4.5 Editorial Workflow

Figure 4.2 illustrates the publication lifecycle of a content item within the system.

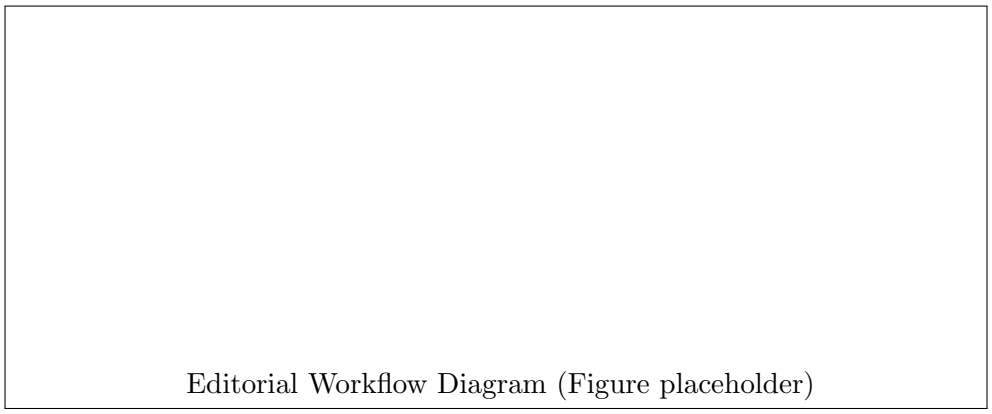

Figure 4.2: Publication workflow state machine.

Content transitions between the following states:

Draft Initial state. The content is only visible to its author.

Pending Approval The author submits the content for review. It becomes visible to Editors.

Published An Editor approves the content; it becomes publicly accessible.

Rejected An Editor rejects the content, optionally providing revision comments. The author may edit and resubmit.

Editors and Administrators bypass the approval step and may publish content directly.

Chapter 5

Data Model

5.1 Entity-Relationship Diagram

Figure 5.1 presents the conceptual Entity-Relationship (ER) diagram of the Diário LX system, capturing the main entities and their associations.

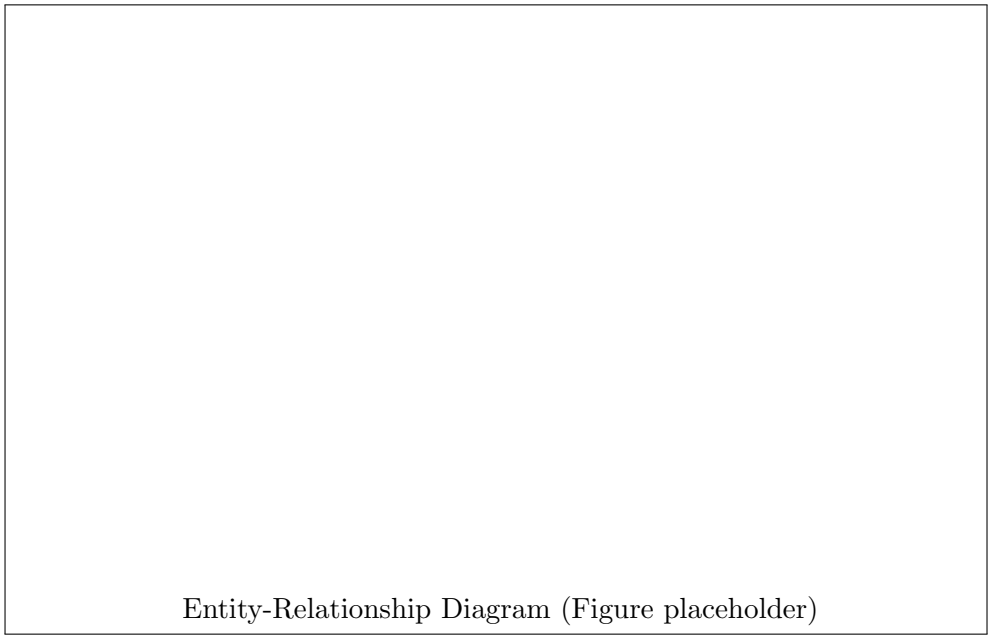

Figure 5.1: Conceptual Entity-Relationship diagram.

5.1.1 Relationship Overview

The main entities and their relationships are described below.

User

Represents an authenticated system user. A user has a role (*Admin*, *Editor*, or *Contributor*) and a function (*Journalist* or *Photographer*). A user may author multiple content items; a content item may have multiple authors.

Content

The central entity of the system. A content item has a type (Article, Podcast, Video, or Photo Essay), a publication state, and references to its authors, categories, and tags. The body of a text article is stored as structured rich text, supporting inline images, subtitles, and highlights.

Category and Subcategory

Content is classified into a two-level category hierarchy. Each category has a name and an associated colour. A content item belongs to exactly one category (and optionally one subcategory).

Tag

Tags provide cross-cutting thematic classification. A content item may be associated with multiple tags; each tag has a dedicated listing page on the public site.

Comment

Readers (if registered) may submit comments on article pages. Comments are moderated by Editors and Administrators.

5.2 Physical Data Model

The physical data model maps the conceptual ER diagram to a relational schema in PostgreSQL. Table 5.1 lists the main database tables and their purpose.

Table 5.1: Main database tables.

Table	Description
`users`	System users (journalists, editors, admins)
`content`	All content items (articles, podcasts, videos, photo essays)
`content_authors`	Many-to-many: content items and their authors
`categories`	Editorial categories with associated colours
`subcategories`	Subcategories linked to parent categories
`tags`	Thematic tags
`content_tags`	Many-to-many: content items and tags
`comments`	Reader comments on published articles
`media`	Metadata for uploaded media files

5.2.1 Key Design Decisions

- **Unified content table**: All content types (articles, podcasts, videos, photo essays) share a common `content` table with a `type` discriminator column. Type-specific attributes are stored in separate extension tables.

- **State machine in the database**: The publication state of a content item is stored as an enumerated column in the `content` table, enforcing valid transitions at the application layer.
- **Media metadata separation**: Binary media files are not stored in the database; only their metadata (URL, size, MIME type) is persisted in the `media` table, with the actual files residing in the dedicated media storage system.

Chapter 6

Backend

6.1 Technology

The backend is developed in **Kotlin** using the **Spring Boot** framework. Kotlin was chosen for its conciseness, null-safety guarantees, and full interoperability with the JVM ecosystem. Spring Boot provides a mature foundation for building REST APIs, including dependency injection, declarative configuration, and robust support for data access and security.

6.2 Organization

The backend follows a layered architecture, inspired by the hexagonal (ports and adapters) pattern. The main layers are:

The backend is organised into the following modules:

HTTP Defines the REST controllers that expose the API endpoints, each responsible for a bounded domain area (e.g., `ContentController`, `UserController`, `CategoryController`). Also contains the authentication interceptor, argument resolvers, and Problem+JSON error handling.

Services Contains the business logic, orchestrating operations across repositories and enforcing domain rules such as publication workflow transitions and permission checks.

Domain Defines the core domain entities and value objects (e.g., `Content`, `User`, `PublicationState`). This module has no dependencies on any other module, ensuring the domain model remains isolated from infrastructure concerns.

Repository Defines the repository interfaces through which the service layer accesses persistent data. The concrete implementation, `Repository JDBI`, implements these interfaces using JDBI, mapping SQL result sets to domain objects. This abstraction allows the persistence layer to be replaced without affecting business logic.

Storage Defines the storage interface for multimedia files. The concrete implementation, `Storage S3`, integrates with SeaweedFS [3] for binary file storage using the AWS S3

SDK [4], as SeaweedFS exposes an S3-compatible API. SeaweedFS was chosen as it can be self-hosted at no cost, which was a requirement given the project's academic and non-commercial nature.

Host The root composition module, responsible for assembling the full application. It contains the Spring Boot entry point, all bean definitions, and the application configuration (database connection, storage credentials, server settings). No business logic resides here — its sole purpose is wiring all modules together and bootstrapping the application.

The external systems used by the backend are **PostgreSQL**, accessed via the JDBI repository implementation, and **SeaweedFS**, accessed via the S3 storage implementation.

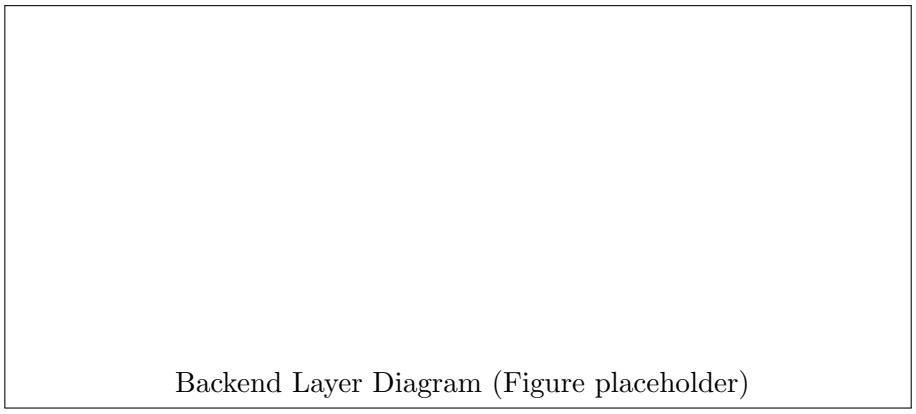

Figure 6.1: Backend layered architecture.

6.3 HTTP API Endpoints

The API is organised around the following resource groups:

- `/api/v1/auth` — Authentication (login, logout, token refresh)
- `/api/v1/users` — User management (CRUD, role assignment)
- `/api/v1/content` — Content management (create, edit, submit, approve, reject, publish, archive, delete)
- `/api/v1/categories` — Category and subcategory management
- `/api/v1/tags` — Tag management
- `/api/v1/comments` — Comment submission and moderation
- `/api/v1/search` — Full-text keyword search

6.4 Authentication

Authentication is implemented using an opaque token mechanism. Upon successful login, a cryptographically random token is generated and stored in a `sessions` table in the database, associated with the authenticated user. The token is then issued to the client as an `HttpOnly` cookie, preventing client-side JavaScript from accessing it and mitigating XSS attacks.

On each subsequent request, the token is extracted from the cookie and looked up in the database to retrieve the associated user and validate the session. If the token is not found or the session has expired, the request is rejected with `401 Unauthorized`.

During the development of this system, the use of JSON Web Tokens (JWT) was evaluated as an alternative. JWT would allow stateless validation of the token without a database lookup, with the user's identity and role encoded directly in the signed token. However, given the nature of the system — a single backend serving a small editorial team — the benefits of JWT did not justify the added complexity. In particular, token revocation would require an additional revoked tokens table, partially negating the stateless advantage. The opaque token approach with a sessions table provides immediate revocation, simpler reasoning about session state, and is well suited to the scale of this system.

6.4.1 AuthenticationInterceptor

The `AuthenticationInterceptor` implements Spring's `HandlerInterceptor` interface, running before each request reaches a controller via the `preHandle` method. It is responsible for three sequential checks:

1. **Token extraction**: the interceptor reads the session token from the request cookies. If the endpoint requires authentication — determined by the presence of `@RequireRole`, or an `AuthenticatedUser` parameter — and no token is found, the request is rejected with `401 Unauthorized`.

2. **Account status**: if a valid token is found but the associated user account is deactivated, the request is rejected with a `403 Forbidden` Problem+JSON response.

3. **Role verification**: if the endpoint is annotated with `@RequireRole`, the authenticated user's role is compared against the required role. A mismatch results in a `403 Forbidden` response.

If all checks pass, the authenticated user is injected into the request context via `AuthenticatedUserArgume` making it available as a parameter in controller methods. Endpoints without any authentication annotation are passed through without restriction.

6.5 Authorization

Role-based access control (RBAC) is enforced at the HTTP layer via Spring Security. Each endpoint is annotated with the required minimum role. The three roles (*Contributor*, *Editor*, *Admin*) form a hierarchical permission model in which each role inherits the permissions of the roles below it.

6.6 Content Workflow

The publication workflow is enforced by the service layer. Valid state transitions are:

- *Draft* → *Pending Approval* (Contributor: submit for review)
- *Draft* → *Published* (Editor/Admin: direct publish)
- *Pending Approval* → *Published* (Editor/Admin: approve)
- *Pending Approval* → *Rejected* (Editor/Admin: reject with comment)
- *Rejected* → *Draft* (Contributor: edit and resubmit)

Invalid transitions are rejected with an appropriate HTTP error response.

6.7 Error Handling

Error responses follow the **Problem Details for HTTP APIs** standard [5], defined in RFC 9457. All error responses return a `application/problem+json` content type with a JSON body containing the following fields:

- `type` — a URI identifying the problem type.
- `title` — a short, human-readable summary.
- `status` — the HTTP status code.
- `detail` — a human-readable explanation specific to the occurrence.

This approach provides a consistent and machine-readable error format across all API endpoints, simplifying error handling on the client side.

6.8 Database Access

Data access is handled through JDBI, a lightweight SQL convenience library for the JVM. Unlike ORM-based approaches, JDBI provides direct control over SQL queries, mapping result sets to domain objects explicitly. This choice favours simplicity and transparency of database interactions over the abstraction overhead of a full ORM.

6.9 Tests

Backend testing is addressed in Chapter 8.

Chapter 7

Frontend

7.1 Technology

The frontend is developed as a single-page application (SPA) using **React** with **TypeScript**. React's component-based model facilitates the construction of reusable UI elements and enables clear separation between the public website and the backoffice interface. TypeScript was adopted to improve code maintainability and reduce runtime errors through static type checking.

7.1.1 Libraries

The following libraries were adopted:

- **React Router** — client-side routing, enabling slug-based navigation (e.g. `/politica/orcamento-estad`
- **Fetch API** — HTTP communication with the backend API [7].
- **TipTap** — rich text editor framework used to implement the block-based article editor in the backoffice [8].
- **Bootstrap** — responsive CSS framework used for layout structuring and interface styling [9].
- **Lucide React** — icon library used throughout the interface to provide consistent and lightweight SVG-based icons [10].

7.2 Organization

The project is structured as follows:

```
src/
  components/       # Reusable UI components
  pages/            # Page-level components (public site)
  backoffice/       # Backoffice-specific pages and components
```

```
hooks/            # Custom React hooks
services/         # API communication layer
context/          # React context providers (auth, theme)
types/            # TypeScript type definitions
utils/            # Utility functions
```

7.3 Navigation

The application uses client-side routing based on React Router. URL slugs follow the pattern `/<category>/<article-slug>`, as required by the specification. Category and tag listing pages are dynamically generated based on data fetched from the API.

7.4 Authentication

The frontend manages authentication state using a React context provider. On successful login, the JWT token is stored (e.g. in memory or a secure cookie) and attached to all subsequent API requests. Protected routes redirect unauthenticated users to the login page.

7.5 Public Website

7.5.1 Homepage

The homepage presents four featured articles and the four most recent articles, fetched from the API on page load. The main navigation menu provides access to all editorial sections and institutional pages.

7.5.2 Article Page

Each article is rendered on a dedicated page at its canonical URL. The page displays the full article content, a comment section, and four related articles.

7.5.3 Category and Tag Pages

Category pages display one featured article followed by remaining articles in reverse chronological order. Tag pages list all articles associated with the tag.

7.5.4 Podcast Page

7.6 Backoffice

7.6.1 Content Management

7.6.2 User Management

7.6.3 Category and Tag Management

7.7 Responsive Design

The interface is fully responsive, adapting to mobile, tablet, and desktop viewports as required. Layout breakpoints follow standard responsive design conventions.

7.8 Tests

Frontend testing is addressed in Chapter 8.

Chapter 8

Tests

This chapter describes the testing strategy adopted for the Diário LX platform, covering both backend and frontend components.

8.1 Backend Tests

8.1.1 Unit Tests

Unit tests target the service layer, validating business logic in isolation from the database and HTTP layer. The publication workflow state machine is a primary focus, ensuring that invalid transitions are correctly rejected and that role-based permission checks behave as expected.

8.1.2 Integration Tests

Integration tests validate the full request-response cycle of the API, including database interactions. A dedicated test database instance is used to avoid interference with development data.

8.2 Frontend Tests

Frontend end-to-end testing is implemented using Playwright [11], a browser automation framework that supports testing across Chromium, Firefox, and WebKit. Playwright tests simulate real user interactions against the running application, covering the main workflows of both the public website and the backoffice.

8.3 Manual Testing

In addition to automated Playwright tests, exploratory manual testing was carried out to validate edge cases and visual behaviour not easily captured by automated scripts, including responsive layout across mobile and desktop viewports.

Chapter 9

Conclusion

9.1 Accomplished Work

This beta report documents the progress achieved in the development of the Diário LX platform. The following components have been implemented or are in an advanced state of development:

- System architecture and technology stack definition.
- Database schema design and initial migration scripts.
- Backend API: authentication, user management, content CRUD, and editorial workflow endpoints.
- Frontend: application structure, routing, authentication flow, and initial page implementations.

9.2 Challenges

Several challenges were encountered during the beta phase:

- **Rich text editor integration**: Implementing a rich text editor that supports inline images, subtitles, and highlights within the article body proved more complex than initially anticipated.
- **Media storage**: The decision regarding the multimedia storage solution depends on infrastructure availability from IPL, which introduces an external dependency on the project timeline.
- **Editorial workflow**: Correctly modelling and enforcing the full set of state transitions and permission rules required careful design of both the backend and frontend layers.

9.3 Future Work

The following items remain to be addressed before the final version:

- Complete implementation of all frontend pages and backoffice screens.
- Finalise media storage solution and integration.
- Implement comment submission and moderation.
- Complete the responsive design across all viewports.
- Expand automated test coverage (unit and integration).
- Deploy the application to the IPL server infrastructure.
- Address optional requirements: scheduled publication, advanced search filters, and reader account registration.

References

[1] Megan Le Masurier. What is slow journalism? *Journalism Practice*, 9(2):138–152, 2015.

[2] WordPress.org. Wordpress features and market share, 2024. [Online; accessed May 2026].

[3] Chris Bai. Seaweedfs — a fast distributed storage system, 2024. [Online; accessed April 2026].

[4] Amazon Web Services. Aws sdk for java 2.x, 2024. [Online; accessed April 2026].

[5] M. Nottingham, E. Wilde, and S. Dalal. Problem details for HTTP APIs, 2023. RFC 9457. [Online; accessed May 2026].

[6] React Router. React router documentation, 2024. [Online; accessed March 2026].

[7] Mozilla Developer Network. Fetch api, 2024. [Online; accessed March 2026].

[8] TipTap. Tiptap editor, 2024. [Online; accessed April 2026].

[9] Bootstrap Team. Bootstrap, 2024. [Online; accessed March 2026].

[10] Lucide. Lucide icons, 2024. [Online; accessed April 2026].

[11] Microsoft Corporation. Playwright — fast and reliable end-to-end testing for modern web applications, 2024. [Online; accessed April 2026].